

DEREXI: POLICY PILOT

Symmetric Cryptography

AES-ECB • AES-CBC • Secure Key Management • ChaCha20

OBJECTIVE

This lab implements the exact symmetric-cryptography elements required by the course rubric: AES encryption in ECB and CBC modes, secure key-generation and management practices, one stream cipher, and documented security considerations. All messages, keys, IVs, and nonces are fictional classroom values; the production DeRexi application remains untouched.

AES: TWO MODES, TWO SECURITY STORIES

AES-256 IMPLEMENTATION

```
key = os.urandom(32)           # 256-bit key
encryptor = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
ciphertext = encryptor.update(padded) + encryptor.finalize()
decryptor = Cipher(algorithms.AES(key), modes.ECB()).decryptor()
iv = os.urandom(16)           # fresh CBC IV
encryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).encryptor()
ciphertext = encryptor.update(padded) + encryptor.finalize()
verified = recovered == plaintext
```

01 ECB

Working encryption/decryption example. Included because the rubric requires it, but repeating plaintext blocks can reveal patterns.

02 CBC

Uses a fresh random IV. The same plaintext encrypted again with a new IV produces different ciphertext.

03 Verification

Both modes decrypt back to the original fictional policy text and return PASS.

SECURITY INTERPRETATION

ECB encrypts each block independently, so identical plaintext blocks can become identical ciphertext blocks. CBC chains blocks together and begins with an initialization vector (IV), which hides repeated structure more effectively. The IV is not secret, but it must be fresh and unique for each encryption under the same key. CBC alone still provides confidentiality, not authentication; modern production systems should prefer authenticated encryption.

DEPENDENCY BOUNDARY

PyCA cryptography was installed in the DeRexi virtual environment and recorded in requirements-dev.txt for classroom use only. The production requirements file and production application logic were left unchanged.

PAGE 1 RESULT

AES-ECB: PASS • AES-CBC: PASS • Fresh-IV behavior confirmed • Production caveats documented

DEREXI: POLICY PILOT

Key Management & Stream Cipher

Secure randomness • Least privilege • ChaCha20 • Nonce safety

KEY MANAGEMENT

The lab generates 256-bit keys from the operating system cryptographically secure random number generator. It explicitly separates demo keys from production practice: real keys belong outside source code, access should follow least privilege, and keys should be rotated after scheduled events or suspected exposure.

KEY-MANAGEMENT EVIDENCE

```

aes_key = generate_aes_key()
stream_key = generate_aes_key()
key_fingerprint = hashlib.sha256(aes_key).hexdigest()[0:16]
Store keys in secrets manager / KMS / HSM
Never commit real keys to Git
Limit access using least privilege
Rotate keys when necessary

```

CHACHA20 EVIDENCE

```

key = generate_aes_key()
nonce = os.urandom(16)
encryptor = Cipher(algorithms.ChaCha20(key, nonce),
    None).encryptor()
ciphertext = encryptor.update(plaintext) +
    encryptor.finalize()
decryptor = Cipher(algorithms.ChaCha20(key, nonce),
    None).decryptor()
recovered = decryptor.update(ciphertext) +
    decryptor.finalize()

```

STREAM-CIPHER SECURITY

ChaCha20 encrypts a byte stream using a key and nonce. The lab goes beyond a basic round-trip test by intentionally reusing the same key and nonce for two messages, demonstrating that the keystream cancels when the ciphertexts are XORed. This proves why nonce reuse is dangerous. The script also notes that authenticated ChaCha20-Poly1305 is preferred in modern production systems.

EXECUTION EVIDENCE

```

- Real keys must never be committed to Git.
- Production keys belong outside source code (secrets manager,
  KMS, protected environment config, or hardware-backed storage).
- Access should follow least privilege.
- Keys should be rotated when necessary (schedule, staff change,
  or suspected exposure).

Stream cipher:
- Nonces must not be reused with the same key.
- Modern authenticated encryption should be preferred when
  available (e.g. ChaCha20-Poly1305 over bare ChaCha20).

=== SUMMARY ===
AES-ECB encryption/decryption: PASS
AES-CBC encryption/decryption: PASS
Secure random key generation: PASS
Stream cipher encryption/decryption: PASS

DeRexi symmetric cryptography lab completed.
adult@DAEDMAC1003 Kiona-629 %

```

01 Stream cipher

ChaCha20 decrypts correctly:
PASS. Nonce-reuse risk is
demonstrated.

02 Key generation

Secure random key generation:
PASS. No production key is
hard-coded.

03 Overall result

ECB PASS • CBC PASS • Stream cipher
PASS • rubric requirements satisfied.